



Instituto Politécnico Nacional
Escuela Superior de Cómputo



Ingeniería en Inteligencia Artificial, Algoritmos bioinspirados
Sem: 2025-1, 5BM1, Fecha: 6-Enero-2025

LA HORMIGA DE LANGTON Y LA INTERACCIÓN ENTRE DIFERENTES HORMIGUEROS

Presenta

Angeles López Erick Jesse¹

Disponible en:

github.com/JesseAngeles/Langton-s-Ant

Resumen

La hormiga de Langton destaca por su simplicidad y su capacidad para generar patrones complejos a partir de reglas simples. En esta practica se explora el comportamiento de una hormiga individual, así como la interacción de múltiples hormigas y hormigueros bajo diferentes configuraciones. Se analizaron reglas clásicas como RL y otras más complejas para identificar la dinámica y la evolución de estos sistemas.

Además, se examinan fenómenos emergentes como la formación de carreteras, ciclos periódicos y comportamientos caóticos.

Palabras clave: Hormiga de Langton, Regla, Evolución, C++, Hormiguero,

¹eangeles11700@alumno.ipn.mx

Índice

1. Introducción	3
2. Marco Teórico	3
2.1. Funcionamiento de la Hormiga de Langton	3
2.2. Clasificación: Agente y Autómata Celular	4
3. Objetivo de la Práctica	5
4. Metodología	6
4.1. Diseño de Software	6
4.2. Comportamiento de una sola Hormiga de Langton	6
4.3. Interacción de múltiples hormigas en el mismo espacio	9
4.3.1. Misma regla	10
4.3.2. Múltiples reglas	13
5. Hormigueros	14
6. Conclusión	19
Referencias	20
7. Código	21
7.1. Lógica	21
7.1.1. Estructuras	21
7.1.2. Hormiga	22
7.1.3. Espacio	25
7.2. Interfaz grafica	30
7.2.1. GUI	30
7.2.2. Grafica	36
7.2.3. Funcionamiento de botones	37

1. Introducción

La hormiga de Langton, creada por Chris Langton en 1986, es un modelo computacional sencillo que genera comportamientos emergentes y patrones complejos a partir de reglas simples. Originalmente, la hormiga sigue dos reglas básicas que dan lugar a un comportamiento caótico e impredecible. Este modelo ejemplifica estructuras del orden al caos, y del caos al orden [1].

En esta práctica, se explora el comportamiento de la hormiga de Langton bajo diferentes condiciones, como cambios en las reglas, la interacción de múltiples hormigas en un mismo espacio y el establecimiento de colonias. Nuestro objetivo es analizar las dinámicas emergentes y las densidades de los estados en el espacio. Para ello, se diseñó un software específico que permite realizar simulaciones personalizadas y graficar los resultados.

2. Marco Teórico

2.1. Funcionamiento de la Hormiga de Langton

La hormiga de Langton se desplaza sobre una cuadrícula bidimensional (teóricamente infinita) en la que cada celda puede tener un estado de 0 o 1 [2]. Sigue dos reglas simples:

1. Si la hormiga está en una celda con estado 0:
 - Cambia el estado de la celda a 1.
 - Gira 90° a la derecha y avanza.
2. Si la hormiga está en una celda con estado 1:
 - Cambia el estado de la celda a 0.
 - Gira 90° a la izquierda y avanza.

Esta configuración inicial se conoce como la regla **RL**, donde:

- El tamaño de la regla define el número de estados consecutivos, en este caso $\{0, 1\}$.
- **R** y **L** indican giros a la derecha (**Right**) e izquierda (**Left**), respectivamente.
- El nuevo estado es el consecutivo en el ciclo. Por ejemplo, del estado 0 pasa al estado 1, y del estado 1 regresa al estado 0.

Bajo esta estructura, pueden definirse múltiples reglas, como la regla **LLR**, que sigue estas instrucciones:

- Si el estado es 0, gira a la izquierda.
 - Si el estado es 1, gira a la izquierda.
 - Si el estado es 2, gira a la derecha.
- Sin olvidar que del estado 0 pasa al 1, del 1 al 2 y del 2 al 0 nuevamente.

2.2. Clasificación: Agente y Autómata Celular

La hormiga de Langton puede ser catalogada tanto como un agente como un autómata celular:

- **Agente:** Es una entidad autónoma que toma decisiones individuales basadas en ciertas reglas [3], lo que coincide con el comportamiento definido anteriormente de la hormiga.
- **Autómata celular:** Un sistema dinámico compuesto por celdas que cambian de estado según una vecindad y una función de transición [4]. Aunque la definición no parece encajar perfectamente, el comportamiento de la hormiga puede replicarse definiendo estados y reglas específicas.

Por ejemplo, para una sola hormiga **RL**, el autómata celular equivalente se define de la siguiente manera:

- **Dimensión:** Plano bidimensional.
- **Estados:** Se agregan 4 estados por cada uno de los estados originales $\{0, 1\}$, para incluir información sobre la dirección de la hormiga: $\{0, 0_U, 0_D, 0_L, 0_R, 1, 1_U, 1_D, 1_L, 1_R\}$, donde el subíndice indica la dirección.
- **Vecindad:** Vecindad de Neumann (los cuatro vecinos ortogonales más cercanos y la celda central).
- **Función de transición:** Las reglas incluyen:
 - Si una celda contiene una hormiga, su estado cambia al estado sin hormiga:

$$c(t) = x_y \rightarrow c(t+1) = x, \forall x \in \text{Estados iniciales}$$

- Si una celda vecina contiene una hormiga mirando hacia ella, adquiere el estado correspondiente con la nueva dirección. Por ejemplo, a continuación se define el comportamiento de una celda si su vecina izquierda tiene una hormiga mirando a la derecha:

$$c_{(-1,0)}(t) = x_R, c(t) = 0 \rightarrow c(t+1) = 1_D, \forall x \in \text{Estados iniciales}$$

$$c_{(-1,0)}(t) = x_R, c(t) = 1 \rightarrow c(t+1) = 0_U, \forall x \in \text{Estados iniciales}$$

3. Objetivo de la Práctica

El objetivo de esta práctica es estudiar los siguientes aspectos:

- El impacto de diferentes reglas sobre el comportamiento de las hormigas.
- Las interacciones entre múltiples hormigas en un mismo espacio.
- La formación, el comportamiento y la interacción entre colonias independientes.

Durante las pruebas, se analizarán y graficarán las densidades de los estados de las celdas. Las simulaciones se llevaron a cabo utilizando un software desarrollado específicamente para esta práctica, detallado en la sección de metodología.

4. Metodología

4.1. Diseño de Software

El software fue desarrollado en C++ utilizando el proyecto *Graphic Mode* [5], que implementa interfaces gráficas mediante la librería SFML. Este programa incorpora múltiples elementos esenciales para la práctica, tales como:

- Creación y manipulación de *frames*.
- Implementación de botones interactivos.
- Campos de texto para la entrada de datos.
- Representación de cuadrículas y gráficas.

Estas herramientas permiten diseñar un entorno visual adecuado para simular, visualizar y analizar el comportamiento de las hormigas de Langton.

4.2. Comportamiento de una sola Hormiga de Langton

La hormiga de Langton, en su versión tradicional, opera bajo la regla **RL**, definida de la siguiente manera:

- Si el estado de la casilla es 0, la hormiga gira a la derecha.
- Si el estado de la casilla es 1, la hormiga gira a la izquierda.

Este sencillo conjunto de reglas produce dos dinámicas características:

1. Un comportamiento inicial caótico y aparentemente aleatorio.
2. La formación de un patrón cíclico denominado “carretera”, que permite a la hormiga expandirse de forma infinita en una dirección.

En la Figura 1 se muestra el resultado del movimiento de una hormiga tras completar 10,000 pasos. Posteriormente, en la Figura 2, se observa cómo la carretera se desarrolla casi de manera inmediata, permitiendo un crecimiento constante desde ese punto.

Cada gráfico utiliza colores para representar los diferentes estados, las casillas negras ó blancas son áreas no visitadas, es decir, que no poseen un estado asignado.

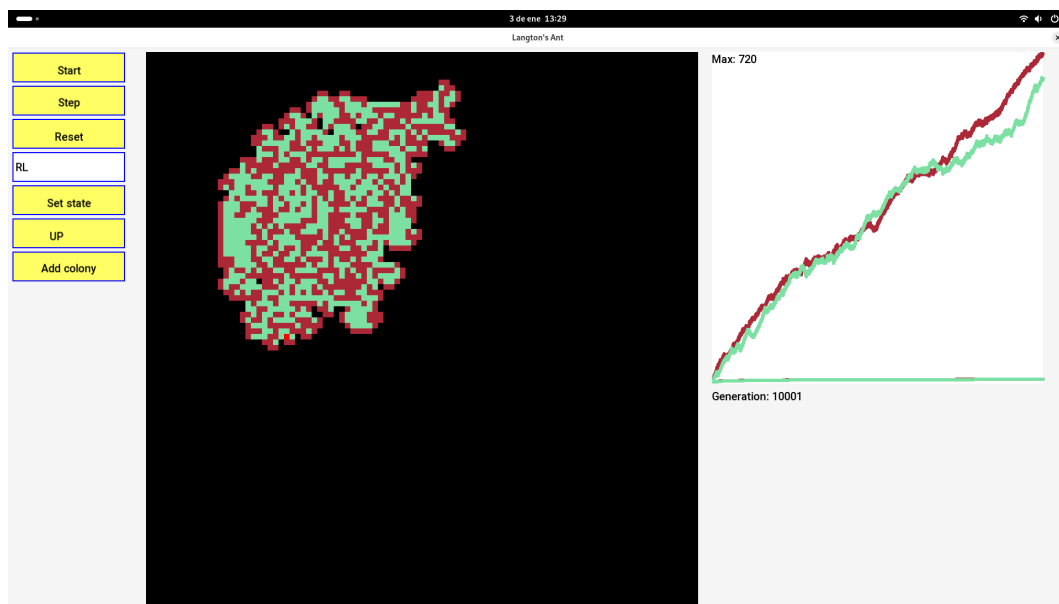


Figura 1: Movimiento inicial de una hormiga bajo la regla **RL** (10,000 pasos).

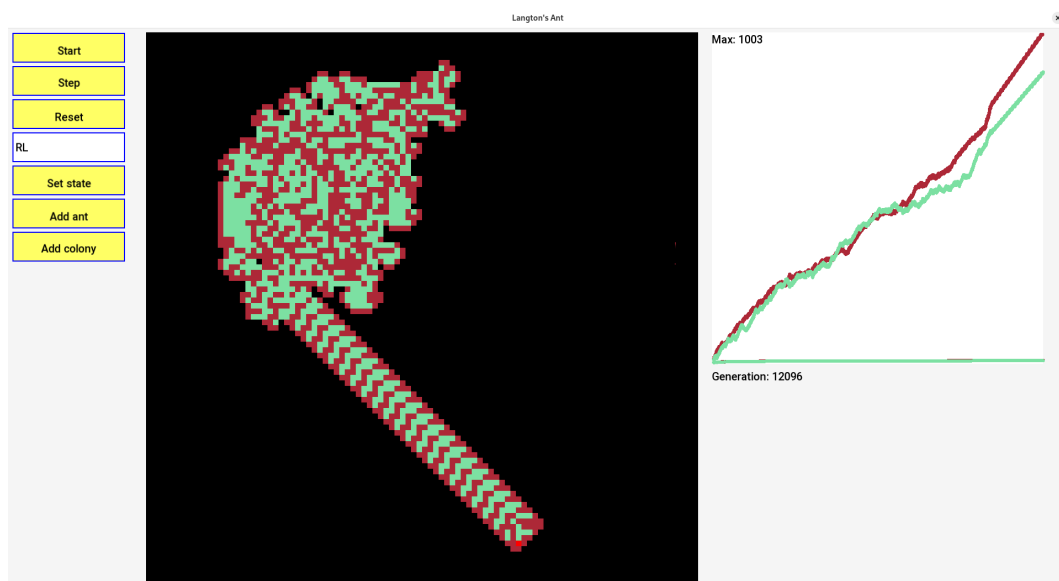
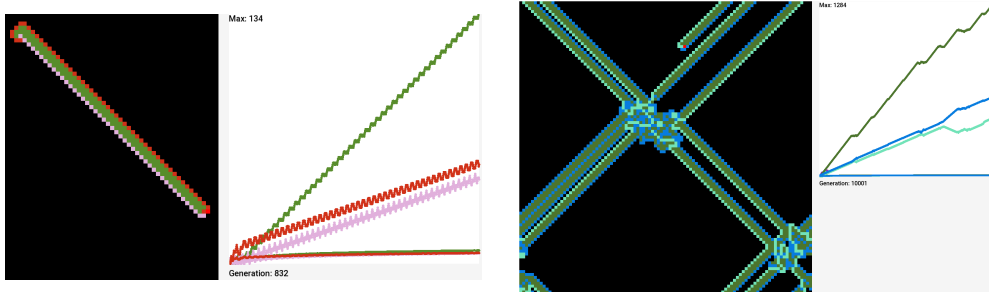
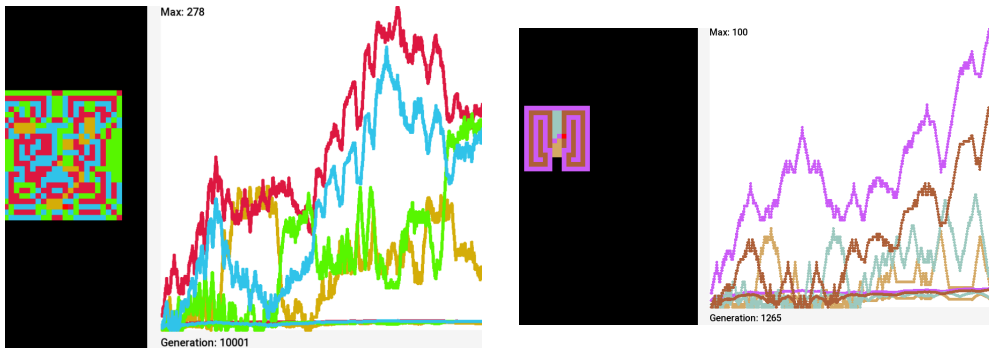


Figura 2: Construcción de la carretera por una hormiga bajo la regla **RL**.

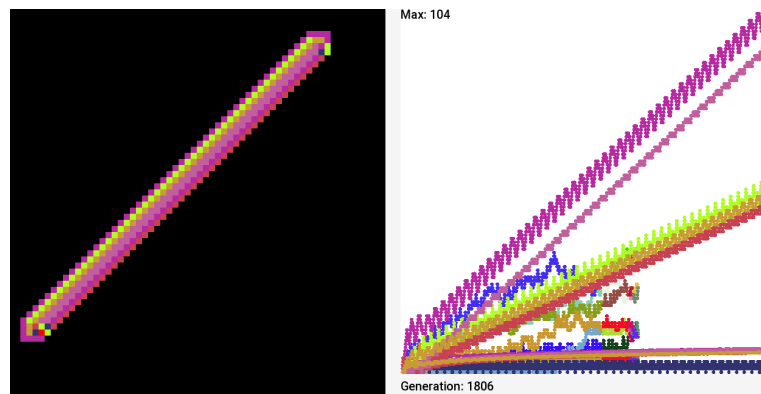
Como se mencionó previamente, las hormigas de Langton pueden operar con diversas reglas, lo que permite modelar y analizar comportamientos variados en un espacio bidimensional. A continuación, en la Figura 3, se presentan configuraciones generadas con distintas reglas. Algunas de estas reglas permiten la creación de carreteras de manera casi inmediata.



La regla **LLR** crea carreteras de forma casi inmediata. Incluso tiene la capacidad de añadir carriles en carreteras existentes. Las perturbaciones en la gráfica representan colisiones con otros elementos en el espacio, aunque, generalmente, se necesitan pocas generaciones para que la hormiga recupere su trayectoria.



La regla **RLLR** se desarrolla en forma cuadrada y no crece hasta que la estructura interna ha cambiado casi por completo. Además, a diferencia de las reglas anteriores, las densidades de estado en esta regla presentan cambios con mayor frecuencia.



La regla **RRRLLL** genera una carretera que tarda el doble de tiempo en formarse en comparación con la regla **LLR**.

Figura 3: Comparación de comportamientos según diferentes reglas.

4.3. Interacción de múltiples hormigas en el mismo espacio

En un espacio con N estados, es posible la coexistencia de múltiples hormigas cuyas reglas tengan la misma longitud, incluso si dichas reglas no coinciden. Sin embargo, surgen dos problemas principales:

1. Si una hormiga tiene menos reglas que los estados disponibles, no tendrá un comportamiento definido al encontrarse en un estado superior al que reconoce.
2. Si una hormiga tiene más reglas que los estados disponibles, generará estados inaccesibles para las demás hormigas, dificultando su interacción con el entorno compartido.

Para abordar estas limitaciones, se proponen las siguientes soluciones:

- Si una hormiga no tiene una regla definida para un estado, avanza en línea recta mientras cambia el estado de la casilla.
- Si una hormiga tiene más reglas que los estados disponibles, sus reglas se restringen al número de estados definidos en el espacio.

Estas soluciones permiten observar nuevos comportamientos, como hormigas que avanzan en línea recta. En la Figura 4, se ilustra el movimiento de una hormiga con la regla RL en un espacio de tres estados.

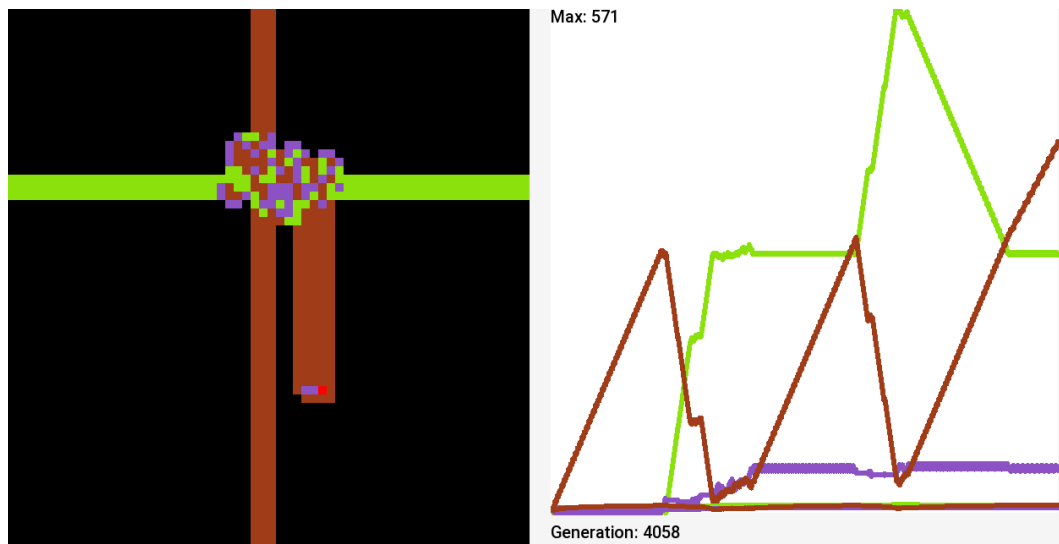


Figura 4: Hormiga con regla RL en un espacio con 3 estados.

4.3.1. Misma regla

El comportamiento pseudoaleatorio característico de la hormiga de Langton se intensifica cuando interactúan múltiples hormigas, como se observa en la Figura 5. Sin embargo, dependiendo de la configuración inicial, pueden emerger dos patrones distintivos.

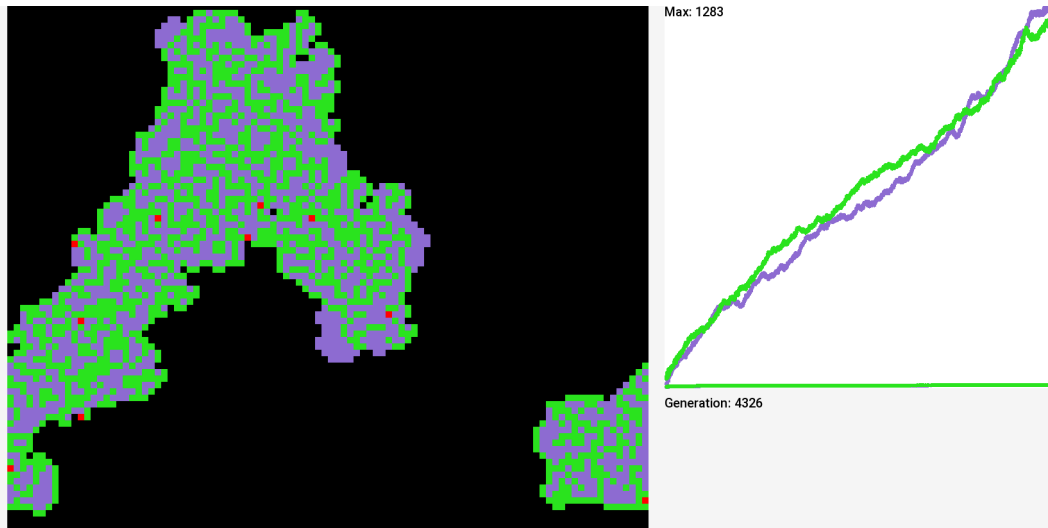


Figura 5: Interacción de múltiples hormigas con regla **RL**.

- **Construcción de carreteras:** El comportamiento caótico inicial puede derivar en la construcción de una carretera antes de alcanzar los 10,000 pasos. Un ejemplo de este fenómeno se ilustra en la Figura 6.

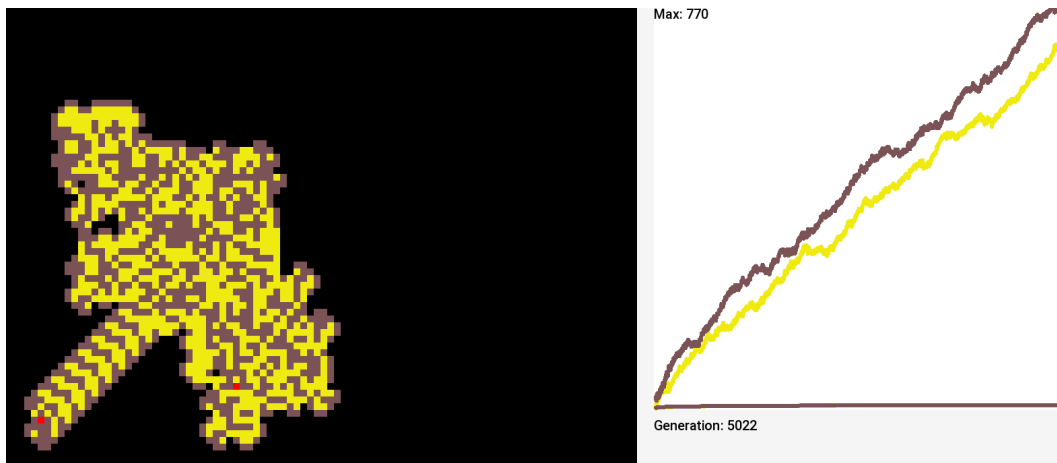
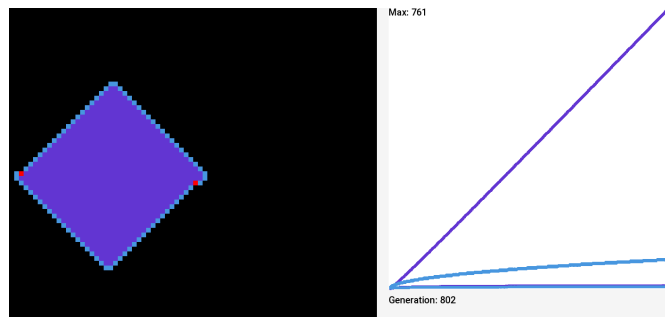


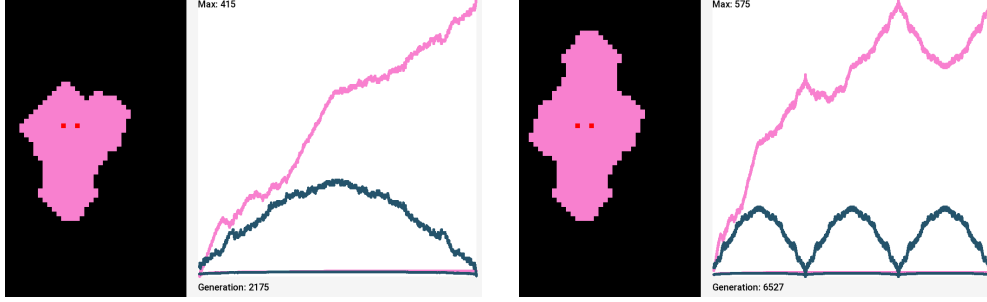
Figura 6: Construcción de una carretera por dos hormigas con regla **RL**.

- **Ciclos:** Colocando dos hormigas en posiciones específicas, es posible generar comportamientos periódicos, como se muestra en la Figura 7. Cabe destacar que obtener un ciclo no es complicado: basta con duplicar una hormiga y posicionarla cerca, ya sea en la misma dirección o en direcciones opuestas.

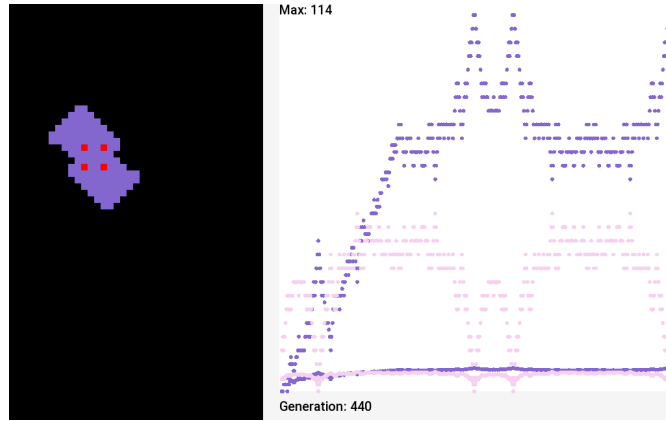
La interacción de múltiples hormigas en el mismo espacio puede provocar colisiones, es decir, situaciones en las que dos hormigas intentan ocupar la misma casilla. Para resolver este problema, se selecciona al azar una hormiga que ocupará la casilla, evitando que dos hormigas compartan la misma posición.



Dos hormigas separadas por $(1, 0)$ y orientadas en la misma dirección son capaces de modificar toda la región accesible del espacio.



Por otro lado, si están separadas por $(+2, 0)$, generan un ciclo con un periodo de 2,175 pasos.



Cuatro hormigas ubicadas en las posiciones iniciales $A_1(0, 0, U)$, $A_2(+3, 0, U)$, $A_3(0, +3, D)$ y $A_4(+3, +3, D)$ generan un ciclo con un periodo de 220 generaciones.

Figura 7: Patrones periódicos generados por múltiples hormigas con regla **RL**.

4.3.2. Múltiples reglas

Como ya se menciono, múltiples hormigas pueden coexistir en el mismo espacio. En la figura 8 se muestran varias de estas configuraciones.

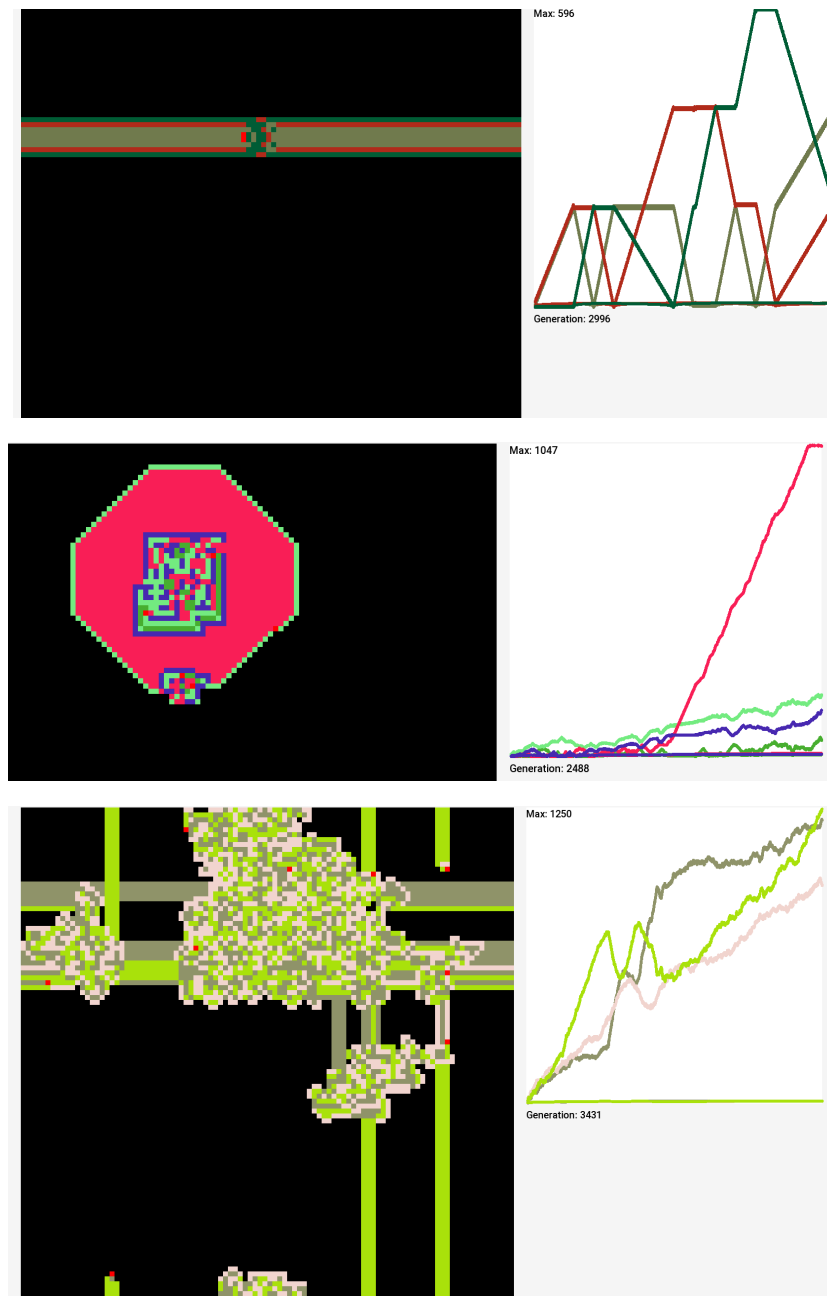


Figura 8: Patrones generados por múltiples hormigas con múltiples reglas.

5. Hormigueros

Al igual que en la naturaleza, un hormiguero se compone de un conjunto de hormigas que comparten un pensamiento colectivo. Este “pensamiento” está representado por la regla que define el comportamiento del grupo de hormigas.

Dado el tamaño requerido de la población únicamente se experimentara con hormigas **RL**.

A continuación se presenta la evolución de cuatro hormigueros con la misma regla, cada uno con una población de 100 hormigas dispersas en un radio de 50 unidades (Distancia de Neumann) mirando en diferentes direcciones. Las claves de los hormigueros serán las siguientes:

- **A:** Superior izquierdo
- **B:** Superior derecho
- **C:** inferior izquierdo
- **D:** Inferior derecho

En la figura 9 se puede observar la generación de un cuadrado azul por parte del hormiguero **D**, lo que significa que se generaron dos hormigas como en la figura 7, es decir, una a un lado de otra con la misma dirección inicial. Este fenómeno dio una ventaja a la densidad de estados azules, representado en la grafica.

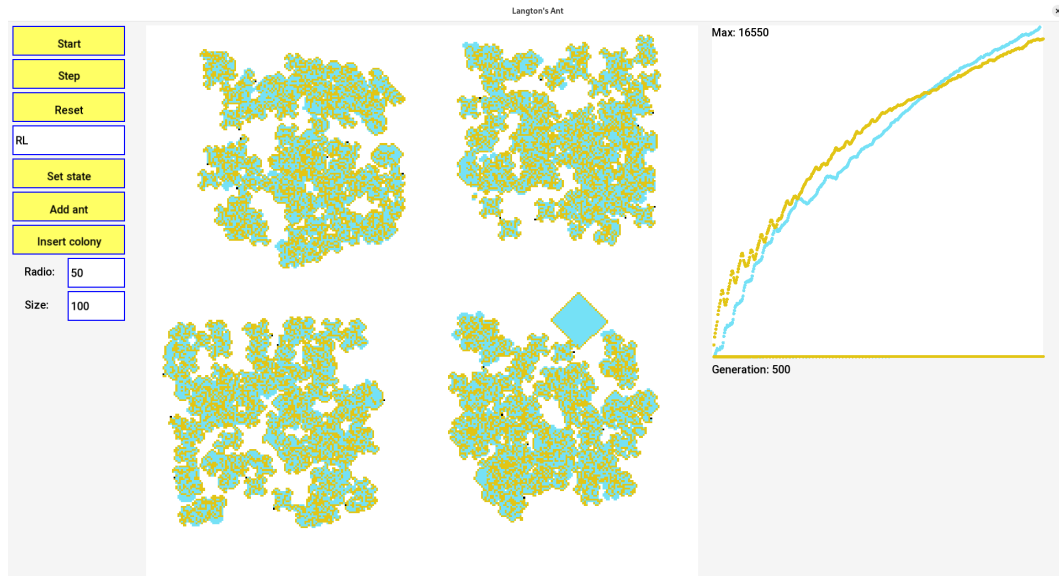


Figura 9: Evolución de hormigueros tras 500 generaciones.

Después de haber transcurrido 1,000 generaciones, las colonias han empezado a construir carreteras como se observa en la figura 10.

- El hormiguero **A** construye una carretera en dirección al hormiguero **B**.
- El hormiguero **C** construye dos carreteras en dirección al hormiguero **A** y **D**.

Ademas, aunque parte del área del rombo azul construido por **D** haya sido alterada, sigue sin representar un cambio significativo en la diferencia de densidades.

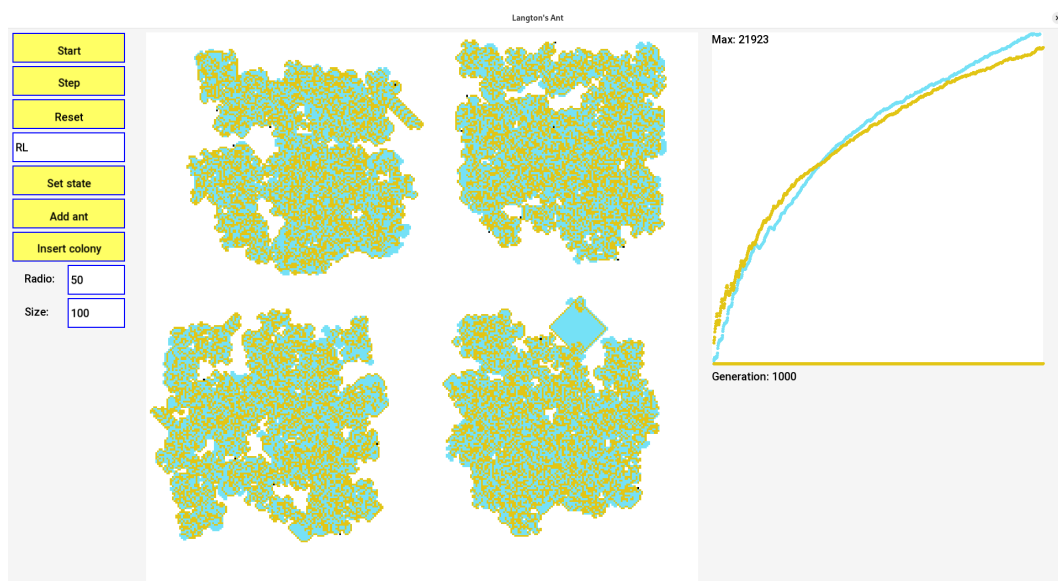


Figura 10: Evolución de hormigueros tras 1000 generaciones.

En la figura 11 se observa la población después de 1,500 generaciones. Todas las carreteras que se se estaban construyendo se vieron interrumpidas por otra hormiga. Esto debido a que las carreteras no solo permiten expandir el área de forma periódica, sino que ademas, permite a otras hormigas viajar a través de ellas de forma mucha mas rápida.

Se han creado nuevas carreteras, la distancia entre hormigueros va disminuyendo, al igual que los espacios no visitados en cada hormiguero.

La curva de la grafica muestra también la velocidad de expansión, adquiriendo una forma logarítmica, lo que implica que entre mas tiempo pase, mas lento sera el dominio del área no visitada. Esto también se aprecia visualmente pues la generación 500 ha tenido mayor crecimiento en comparación con el resto.

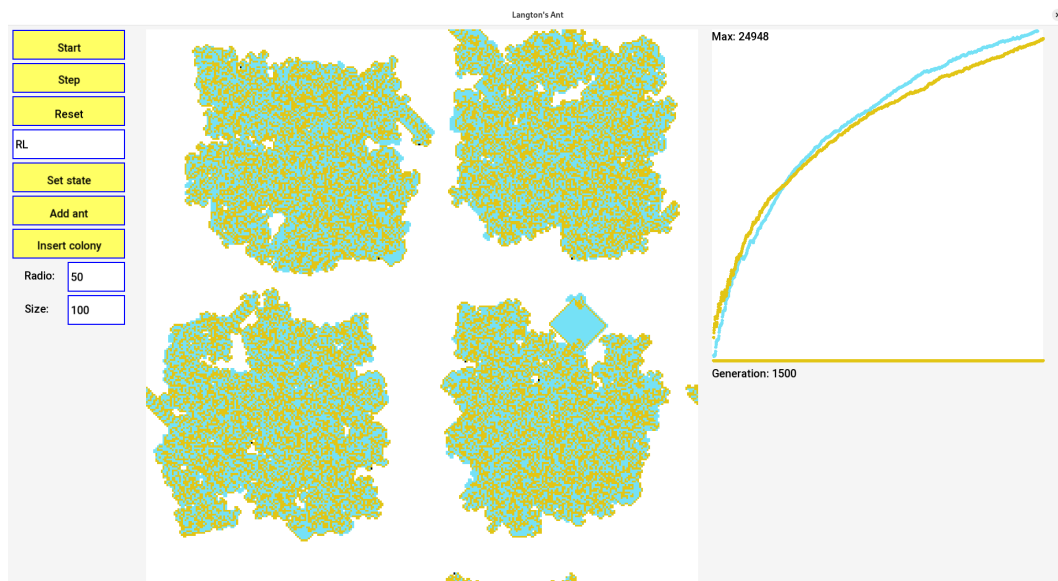


Figura 11: Evolución de hormigueros tras 1,500 generaciones.

En la figura 12 aparece un ligero incremento en la expansión, ya que todos los hormigueros empezaron a construir carreteras. Se está dando la primera conexión entre los hormigueros **C** con **D**, **A** con **B** y **A** con **C**.

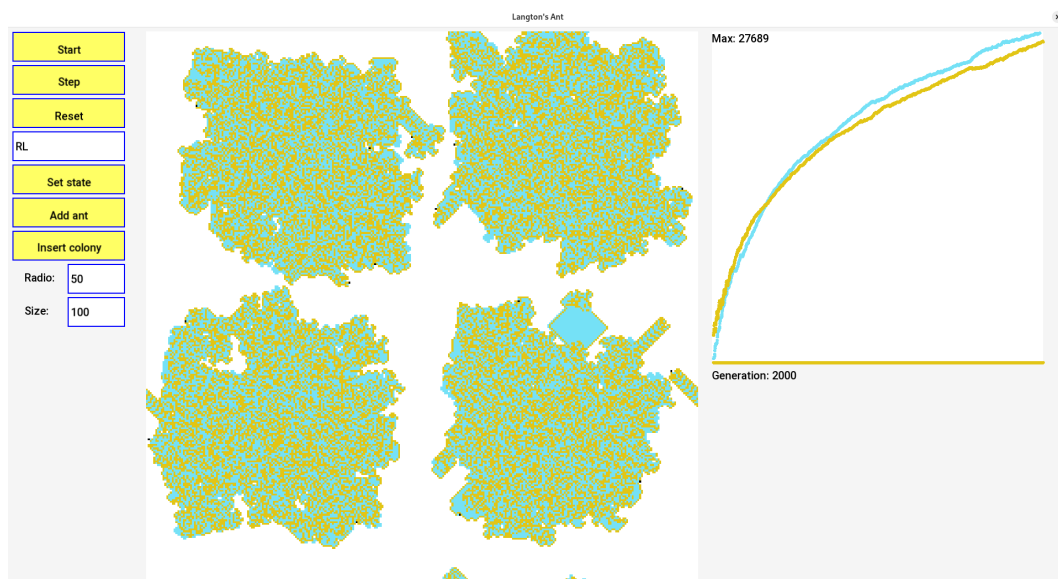


Figura 12: Evolución de hormigueros tras 2,000 generaciones.

Los hormigueros, en la figura 13, están próximos a conectarse mediante carreteras. Parecieran reacios a unirse, pues delimitaron las carreteras por los bordes de otros hormigueros. Un fenómeno completamente aleatorio.

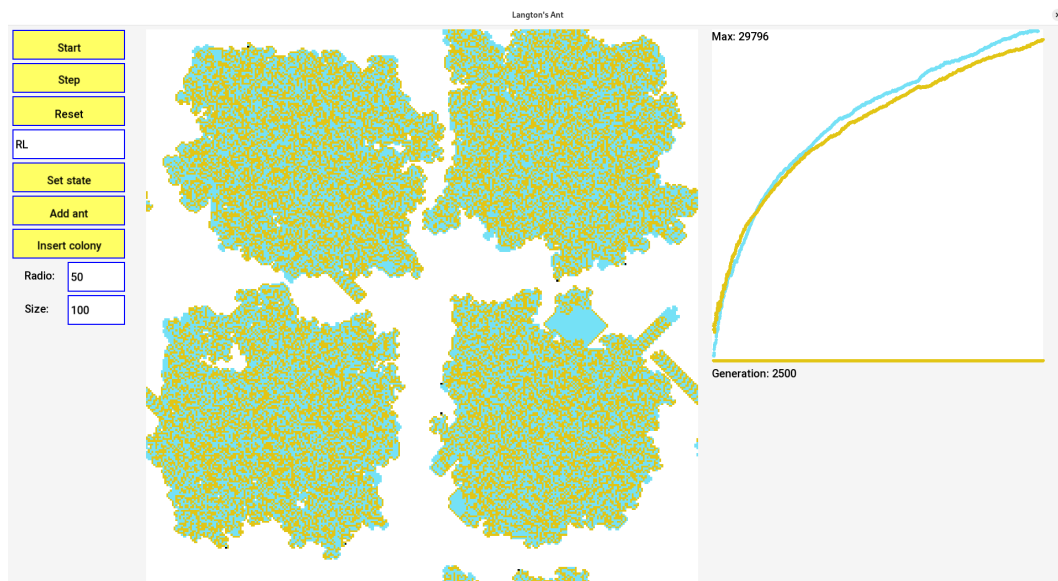


Figura 13: Evolución de hormigueros tras 2,500 generaciones.

Finalmente, en la figura 14 los hormigueros se han unidos. No están completamente conexos, pero sin ningún problema, una hormiga de cualquier hormiguero puede visitar todos los hormigueros mediante las carreteras. Pareciera tener un crecimiento constante desde las ultimas generaciones, pero esto se debe a que, al crear nuevos caminos, las hormigas se mueven con mayor facilidad, pudiendo expandir territorio en otros hormigueros.

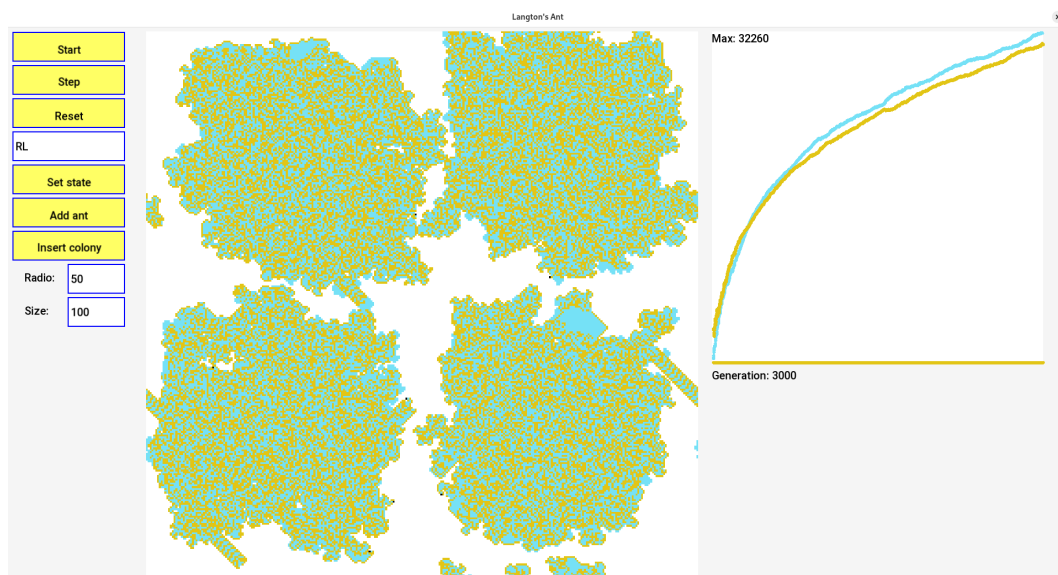


Figura 14: Evolución de hormigueros tras 3,000 generaciones.

En la tabla 1 se compara la expansión de los hormigueros, y como estos disminuyen exponencialmente.

Generación	Espacio	Crecimiento
500	16,550	16,550
1,000	21,923	5,373
1,500	24,948	3,025
2,000	27,689	2,741
2,500	29,796	2,103
3,000	32,360	2,564

Tabla 1: Expansión de hormigueros

6. Conclusión

Cada hormiga, con cada paso que da, es capaz de generar caos en su entorno. Sin embargo, la interacción entre múltiples hormigas puede dar lugar a comportamientos colectivos organizados, como patrones cíclicos o expansivos. Este fenómeno sugiere que pocas hormigas son capaces de trabajar de manera independiente, como las que construyen carreteras, mientras que la mayoría tiende a sincronizarse, como si una hormiga reina dictara las instrucciones o “reglas” para el grupo.

Es fascinante observar cómo, a partir de un universo tan simple basado en dos giros, pueden emerger configuraciones altamente complejas.

Esta práctica representa apenas una exploración superficial del comportamiento de la hormiga de Langton, considerando únicamente sistemas bidimensionales con múltiples reglas en un mismo espacio. Sin embargo, se abre un vasto horizonte de posibilidades si se introducen nuevos elementos, como un sistema tridimensional o espacios no ortogonales, como hexagonales o triangulares. Esto daría lugar a una gran variedad de combinaciones, patrones, comportamientos y, posiblemente, aplicaciones.

Como trabajo futuro queda la interacción cíclica de hormigas con diferentes reglas y la construcción de patrones en escalas mayores, tanto de espacio como de tiempo. Nuevamente una hormiga demuestra que los cambios pequeños dan grandes resultados.

Referencias

- [1] Contributors to Wikimedia projects. “Langton’s ant - wikipedia”. Wikipedia, the free encyclopedia. Accedido el 26 de diciembre de 2024. [En línea]. Disponible: https://en.wikipedia.org/wiki/Langton's_ant
- [2] N. Freire. “La hormiga de Langton, de la simplicidad al caos”. National Geographic España. Accedido el 25 de diciembre de 2024. [En línea]. Disponible: https://www.nationalgeographic.com.es/ciencia/hormiga-langton-simplicidad-caos_22530
- [3] Colaboradores de los proyectos Wikimedia. “Agente (software) - Wikipedia, la enciclopedia libre”. Wikipedia, la enciclopedia libre. Accedido el 25 de diciembre de 2024. [En línea]. Disponible: [https://es.wikipedia.org/wiki/Agente_\(software\)](https://es.wikipedia.org/wiki/Agente_(software))
- [4] Wolfram Research, Inc. “Cellular Automaton – from Wolfram MathWorld”. Wolfram MathWorld: The Web’s Most Extensive Mathematics Resource. Accedido el 25 de diciembre de 2024. [En línea]. Disponible: <https://mathworld.wolfram.com/CellularAutomaton.html>
- [5] J. Angeles. “GitHub - JesseAngeles/Graphic-Mode: Graphic mode using SFML”. GitHub. Accedido el 4 de enero de 2025. [En línea]. Disponible: <https://github.com/JesseAngeles/Graphic-Mode>

7. Código

7.1. Lógica

7.1.1. Estructuras

```
1 struct Cell {  
2     int state;  
3     std::optional<std::shared_ptr<Ant>> ant;  
4     Cell(int state) : state(state), ant(std::nullopt) {}  
5     Cell(int state, std::shared_ptr<Ant>) : state(state),  
6         ant(ant) {}  
};
```

Listing 1: Cell

```
1 enum class Direction {  
2     UP,  
3     LEFT,  
4     RIGHT,  
5     DOWN  
6 };
```

Listing 2: Direction

```
1 struct Position2D {  
2     int x;  
3     int y;  
4     Position2D() = default;  
5     Position2D(int x, int y) : x(x), y(y) {}  
6     bool operator==(const Position2D &other) const { return  
7         x == other.x && y == other.y; }  
8     bool operator<(const Position2D &other) const {  
9         if (x < other.x)  
10            return true;  
11        if (other.x < x)  
12            return false;  
13  
14        if (y < other.y)  
15            return true;  
16        return false;  
17    }  
};
```

Listing 3: Position

```
1 enum class Rule {
2     LEFT,
3     RIGHT
4 };
5
6 inline std::vector<Rule> stringToRules(std::string &rule) {
7     std::vector<Rule> rules;
8     for (const char c : rule)
9         if (c == 'L')
10            rules.push_back(Rule::LEFT);
11        else if (c == 'R')
12            rules.push_back(Rule::RIGHT);
13
14    return rules;
15 }
```

Listing 4: Rule

7.1.2. Hormiga

```
1 class Ant {
2     private:
3         // Attributes
4         Position2D pos;
5         Direction direction;
6
7         Position2D space;
8
9         int colony;
10
11        // Rules
12        std::vector<Rule> rules;
13
14        // Functions
15
16    public:
17        // Constructor
18        Ant(Position2D pos, Position2D space, Direction
19            direction, std::vector<Rule> rules, int colony = 0);
20
21        // Functions
22        void rotate(const int &state);
23        Position2D getNextMove() const;
```

```

23
24 // Overloading
25 bool operator<(<const Ant &other> const;
26
27 // Getters
28 Position2D getPos() const { return pos; }
29 Direction getDirection() const { return direction; }
30 std::vector<Rule> getRules() const { return rules; }
31
32 // Setters
33 void setPos(Position2D &pos) { this->pos = pos; }
34 void setDirection(Direction &direction) { this->
    direction = direction; }
35 void setRules(std::vector<Rule> &rules) { this->rules =
    rules; }
36 void setSpace(Position2D &space);
37
38 // Display
39 void display();
40 };

```

Listing 5: Ant.hpp

```

1 // Constructor
2 Ant::Ant(Position2D pos, Position2D space, Direction
    direction, std::vector<Rule> rules, int colony)
3 : pos(pos), space(space), direction(direction), rules(rules)
    , colony(0) {}
4
5 // Public functions
6 void Ant::rotate(const int &state) {
7     if (state >= rules.size())
8         return;
9
10    Rule rule_to_apply = rules[state];
11
12    if (rule_to_apply != Rule::LEFT) {
13        if (direction == Direction::LEFT)
14            direction = Direction::DOWN;
15        else if (direction == Direction::UP)
16            direction = Direction::LEFT;
17        else if (direction == Direction::RIGHT)
18            direction = Direction::UP;
19        else if (direction == Direction::DOWN)
20            direction = Direction::RIGHT;

```

```

21     } else if (rule_to_apply != Rule::RIGHT) {
22         if (direction == Direction::LEFT)
23             direction = Direction::UP;
24         else if (direction == Direction::UP)
25             direction = Direction::RIGHT;
26         else if (direction == Direction::RIGHT)
27             direction = Direction::DOWN;
28         else if (direction == Direction::DOWN)
29             direction = Direction::LEFT;
30     }
31 }
32
33 Position2D Ant::getNextMove() const {
34     Position2D pos = this->pos;
35
36     if (direction == Direction::UP)
37         pos.x--;
38     else if (direction == Direction::RIGHT)
39         pos.y++;
40     else if (direction == Direction::DOWN)
41         pos.x++;
42     else if (direction == Direction::LEFT)
43         pos.y--;
44
45     pos.x = (pos.x + space.x) % space.x;
46     pos.y = (pos.y + space.y) % space.y;
47
48     return pos;
49 }
50
51 // Overloading
52 bool Ant::operator<(const Ant &other) const {
53     if (pos.x < other.pos.x)
54         return true;
55     if (other.pos.x < pos.x)
56         return false;
57
58     if (pos.y < other.pos.y)
59         return true;
60
61     return false;
62 }
63
64 // Setters
65 void Ant::setSpace(Position2D &space) {

```



```
66     this->space = space;
67     pos.x %= space.x;
68     pos.y %= space.y;
69 }
70
71 // Display
72 void Ant::display() {
73     std::cout << "\n(" << pos.x << "," << pos.y << ") -> ";
74     if (direction == Direction::DOWN)
75         std::cout << "down";
76     else if (direction == Direction::LEFT)
77         std::cout << "left";
78     else if (direction == Direction::UP)
79         std::cout << "up";
80     else if (direction == Direction::RIGHT)
81         std::cout << "right";
82
83     std::cout << ": ";
84
85     for (const Rule &rule : rules)
86         if (rule == Rule::LEFT)
87             std::cout << "L";
88         else if (rule == Rule::RIGHT)
89             std::cout << "R";
90 }
```

Listing 6: Ant.cpp

7.1.3. Espacio

```
1  class Space {
2      private:
3          // Attributes
4          std::vector<std::vector<Cell>> space;
5          std::set<std::shared_ptr<Ant>> ants;
6
7          int states;
8
9          Randomizer randomizer;
10
11         // Functions
12
13     public:
14         // Constructor
```

```

15     Space(int width, int height, int states, std::set<std::
        shared_ptr<Ant>> ants);
16
17     // Functions
18     std::set<Position2D> move();
19     void switchCell(Position2D pos) { space[pos.x][pos.y].
        state = (space[pos.x][pos.y].state + 1) % states; }
20     bool insertAnt(std::shared_ptr<Ant> &ant);
21     void removeAnt(Position2D pos);
22     void clear();
23
24     int insertColony(Position2D center, float radius, int
        population, std::vector<Rule> rules, int colony);
25
26     // Getters
27     std::vector<std::vector<Cell>> getSpace() const { return
        space; }
28     std::set<std::shared_ptr<Ant>> getAnts() const { return
        ants; }
29     int getStates() const { return states; }
30
31     // Setters
32     void setSpace(std::vector<std::vector<Cell>> &space);
33     void setAnts(std::set<std::shared_ptr<Ant>> &ants) {
        this->ants = ants; }
34     void setSpace(int &states) { this->states = states; }
35
36     // Display
37     void displaySpace();
38     void displayAnts();
39     void display();
40 };

```

Listing 7: Space.hpp

```

1 // Constructor
2 Space::Space(int width, int height, int states, std::set<std
    ::shared_ptr<Ant>> ants)
3 : space(width, std::vector<Cell>(height, Cell(-1))), states(
    states), ants(ants), randomizer() {
4     for (const std::shared_ptr<Ant> &ant : ants)
5         this->space[ant->getPos().x][ant->getPos().y].ant = ant;
6 }
7
8 // Private functions

```

```
9
10 // Public functions
11 std::set<Position2D> Space::move() {
12     // Rotar todas las hormigas
13     for (const std::shared_ptr<Ant> &ant : ants) {
14         Position2D pos = ant->getPos();
15         if (space[pos.x][pos.y].state == -1)
16             ant->rotate(0);
17         else
18             ant->rotate(space[pos.x][pos.y].state);
19     }
20
21     // Obtener todas las casillas que seran ocupadas
22     std::map<Position2D, std::vector<std::shared_ptr<Ant>>>
        taken_cells;
23
24     for (const std::shared_ptr<Ant> &ant : ants) {
25         Position2D next_move = ant->getNextMove();
26
27         auto it = taken_cells.find(next_move);
28         if (it != taken_cells.end())
29             it->second.push_back(ant);
30         else
31             taken_cells[next_move] = {ant};
32     }
33
34     // Buscar aquellas casillas que puede ser tomada por dos
        o mas, y seleccionar una al azar
35     std::vector<std::shared_ptr<Ant>> movable_ants;
36     for (std::pair<Position2D, std::vector<std::shared_ptr<
        Ant>>> cell : taken_cells)
37         if (cell.second.size() == 1)
38             movable_ants.push_back(cell.second[0]);
39         else
40             movable_ants.push_back(cell.second[std::round(randomizer
                .generate(0, cell.second.size() - 1))]);
41
42     // Eliminar referencia de ants
43     for (const std::shared_ptr<Ant> &ant : movable_ants)
44         space[ant->getPos().x][ant->getPos().y].ant = std::
            nullopt;
45
46     // Mover movable_ants
47     std::set<Position2D> positions_to_update;
48
```

```

49     for (const std::shared_ptr<Ant> &ant : ants) {
50         Position2D pos = ant->getPos();
51         Position2D new_pos = ant->getNextMove();
52         ant->setPos(new_pos);
53
54
55         if (space[pos.x][pos.y].state == -1)
56             space[pos.x][pos.y].state = (space[pos.x][pos.y].
57                 state + 2) % states;
58         else
59             space[pos.x][pos.y].state = (space[pos.x][pos.y].
60                 state + 1) % states;
61         space[new_pos.x][new_pos.y].ant = ant;
62
63         positions_to_update.insert(pos);
64         positions_to_update.insert(new_pos);
65     }
66
67     return positions_to_update;
68 }
69
70 bool Space::insertAnt(std::shared_ptr<Ant> &ant) {
71     Position2D pos = ant->getPos();
72     if (!space[pos.x][pos.y].ant.has_value()) {
73         ants.insert(ant);
74         space[pos.x][pos.y].ant = ant;
75
76         return true;
77     }
78     return false;
79 }
80
81 void Space::removeAnt(Position2D pos) {
82     if (space[pos.x][pos.y].ant.has_value()) {
83         std::shared_ptr<Ant> ant = space[pos.x][pos.y].ant.
84             value();
85         space[pos.x][pos.y].ant = std::nullopt;
86
87         ants.erase(ant);
88     }
89 }
90
91 void Space::clear() {
92     for (std::vector<Cell> &line : space)
93         for (Cell &cell : line) {

```

```
91         cell.ant = std::nullopt;
92         cell.state = -1;
93     }
94
95     ants.clear();
96 }
97
98 int Space::insertColony(Position2D center, float radius, int
99 population, std::vector<Rule> rules, int colony) {
100     // Create N ants
101     int i = 0, j = 0;
102     int total_count = std::sqrt(8) * std::pow(radius, 2);
103     Position2D current_space(space.size(), space[0].size());
104
105     while (i < population && j++ < total_count) {
106         int x_pos = randomizer.generate(center.x + radius,
107             center.x - radius);
108         int y_pos = randomizer.generate(center.y + radius,
109             center.y - radius);
110         int direction = randomizer.generate(0, 4);
111
112         Direction new_direction;
113         switch (direction) {
114             case 0:
115                 new_direction = Direction::LEFT;
116                 break;
117             case 1:
118                 new_direction = Direction::UP;
119                 break;
120             case 2:
121                 new_direction = Direction::DOWN;
122                 break;
123             case 3:
124                 new_direction = Direction::RIGHT;
125                 break;
126         }
127
128         std::shared_ptr<Ant> new_ant = std::make_shared<Ant>
129             >(Position2D(x_pos, y_pos), current_space,
130                 new_direction, rules, colony);
131
132         if (insertAnt(new_ant))
133             i++;
134     }
```

```
131     return i;
132 }
133
134 // Setters
135 void Space::setSpace(std::vector<std::vector<Cell>> &space)
136 {
137     this->space = space;
138     Position2D new_space(space.size(), space[0].size());
139     for (const std::shared_ptr<Ant> &ant : ants)
140         ant->setSpace(new_space);
141 }
```

Listing 8: Space.cpp

7.2. Interfaz grafica

7.2.1. GUI

```
1 class GUI : public MainWindow {
2     private:
3         // Attributes
4         int direction = 0;
5         int generation_count = 0;
6         int current_colonies = 0;
7         int rule_size;
8         int size = 1;
9         int radio = 1;
10        bool is_running = false;
11        bool is_colony = false;
12
13        std::string current_rule;
14
15        // GUI Components
16        sf::Font font;
17        sf::Text default_text;
18        sf::Text generation_count_text;
19        sf::Text max_generation_text;
20        sf::Text radio_text;
21        sf::Text size_text;
22        std::vector<Color> colors;
23
24        Frame frame;
25        std::shared_ptr<MainGraphic> graphic;
```

```

26     std::shared_ptr<MainGrid> grid;
27
28     std::shared_ptr<TextBoxRule> tb_rule;
29     std::shared_ptr<TextBoxRadio> tb_radio;
30     std::shared_ptr<TextBoxSize> tb_size;
31
32     std::shared_ptr<ButtonStart> btn_start;
33     std::shared_ptr<ButtonStep> btn_step;
34     std::shared_ptr<ButtonReset> btn_reset;
35     std::shared_ptr<ButtonState> btn_state;
36     std::shared_ptr<ButtonAnt> btn_ant;
37     std::shared_ptr<ButtonColony> btn_colony;
38
39     void initDefaultText();
40     void render() override;
41
42     public:
43     // Constructor
44     GUI(const std::shared_ptr<Space> &);
45     void run() override;
46
47     // Logic
48     std::shared_ptr<Space> space;
49
50     void move();
51     void clear();
52     void initColors();
53     Color generateRandomColor();
54
55     // Test functions
56     void testButton() { std::cout << "Test\n"; };
57     void testTextBox(std::string text) { std::cout << "Test:
58         " << text << std::endl; }
59     void testGrid(int x, int y) { std::cout << "Test: (" <<
60         x << "," << y << ")\n"; }
61
62     // Getters
63     int getDirection() const { return direction; }
64     int getGenerationCount() const { return generation_count
65         ; }
66     int getRuleSize() const { return rule_size; }
67     bool isRunning() const { return is_running; }
68     bool isColony() const { return is_colony; }
69     std::string getRule() const { return current_rule; }
70     int getSize() const { return size; }

```

```

68     int getRadio() const { return radio; }
69     int getCurrentColonies() const {return current_colonies;
70     }
71
72     // Setters
73     void setDirection(int direction) { this->direction =
74     direction; }
75     void resetGenerationCount() { generation_count = 0; }
76     void setRuleSize(int rule_size) { this->rule_size =
77     rule_size; }
78     void switchRunning() { is_running = !is_running; }
79     void switchColony() { is_colony = !is_colony; }
80     void setRule(std::string current_rule) { this->
81     current_rule = current_rule; }
82     void setSize(int size) { this->size = size; }
83     void setRadio(int radio) { this->radio = radio; }
84     void setMaxGenerationText(int x) { max_generation_text.
85     setString("Max: " + std::to_string(x)); }
86     void setGenerationCountText() { generation_count_text.
87     setString("Generation: " + std::to_string(
88     generation_count)); }
89     void setButtonAntText(const std::string &content) {
90     btn_ant->setText(content); }
91     void setCurrentColonies(int current_colonies) {this->
92     current_colonies = current_colonies; }
93 };

```

Listing 9: GUI.hpp

```

1 // Constructor
2 GUI::GUI(const std::shared_ptr<Space> &space)
3 : space(space),
4   MainWindow(1920, 1080, "Langton's Ant"),
5   frame(Vector2f(0, 0), Vector2f(1920, 1080), Color(245, 245,
6   245)),
7   graphic(std::make_shared<MainGraphic>(this)),
8   grid(std::make_shared<MainGrid>(this, space->getSpace().size
9   (), space->getSpace()[0].size())),
10  tb_rule(std::make_shared<TextBoxRule>(this, default_text)),
11  tb_radio(std::make_shared<TextBoxRadio>(this, default_text))
12  ,
13  tb_size(std::make_shared<TextBoxSize>(this, default_text)),
14  btn_start(std::make_shared<ButtonStart>(this, default_text))
15  ,
16  btn_step(std::make_shared<ButtonStep>(this, default_text)),

```



```
13 btn_reset(std::make_shared<ButtonReset>(this, default_text))
14 ,
15 btn_state(std::make_shared<ButtonState>(this, default_text))
16 ,
17 btn_ant(std::make_shared<ButtonAnt>(this, default_text)),
18 btn_colony(std::make_shared<ButtonColony>(this, default_text
19 )) {
20     initDefaultText();
21     rule_size = space->getStates();
22
23     tb_rule->setText(default_text);
24     tb_radio->setText(default_text);
25     tb_size->setText(default_text);
26
27     btn_start->setText(default_text);
28     btn_start->setText("Start");
29
30     btn_step->setText(default_text);
31     btn_step->setText("Step");
32
33     btn_reset->setText(default_text);
34     btn_reset->setText("Reset");
35
36     btn_state->setText(default_text);
37     btn_state->setText("Set state");
38
39     btn_ant->setText(default_text);
40     btn_ant->setText("Add ant");
41
42     btn_colony->setText(default_text);
43     btn_colony->setText("Insert colony");
44
45     frame.addChild(graphic);
46     frame.addChild(grid);
47     frame.addChild(tb_rule);
48     frame.addChild(btn_start);
49     frame.addChild(btn_step);
50     frame.addChild(btn_reset);
51     frame.addChild(btn_ant);
52     frame.addChild(btn_state);
53     frame.addChild(btn_colony);
54     frame.addChild(tb_radio);
55     frame.addChild(tb_size);
56
57     addFrame(frame);
```

```
55 }
56
57 // Private functions
58 void GUI::initDefaultText() {
59     if (!font.loadFromFile("./resources/fonts/Roboto-Medium.
60         ttf")) {
61         throw std::runtime_error("No se pudo cargar la
62             fuente.");
63     }
64
65     default_text.setFont(font);
66     default_text.setFillColor(Color::Black);
67     default_text.setCharacterSize(20);
68     default_text.setString("Default Text");
69
70     max_generation_text = default_text;
71     max_generation_text.setPosition(1275, 7);
72     max_generation_text.setString("Max: 0");
73
74     generation_count_text = default_text;
75     generation_count_text.setPosition(1275, 617);
76     generation_count_text.setString("Generation: 0");
77
78     radio_text = default_text;
79     radio_text.setPosition(30, 440);
80     radio_text.setString("Radio:");
81
82     size_text = default_text;
83     size_text.setPosition(30, 500);
84     size_text.setString("Size:");
85 }
86
87 void GUI::render() {
88     window.clear(background_color);
89
90     // Render frames
91     for (const std::shared_ptr<Frame> &frame : frames)
92         window.draw(*frame);
93
94     window.draw(max_generation_text);
95     window.draw(generation_count_text);
96     window.draw(radio_text);
97     window.draw(size_text);
98
99     window.display();
```

```
98 }
99
100 // Public functions
101 void GUI::run() {
102     Clock clock;
103     float elapsed_time = 0.0f;
104
105     while (window.isOpen()) {
106         float delta_time = clock.restart().asSeconds();
107         if (is_running) {
108             elapsed_time += delta_time;
109             if (elapsed_time >= 0.0f && generation_count <=
110                 10000) {
111                 move();
112                 elapsed_time = 0.0f; // Reinicia el tiempo
113                                     acumulado
114             }
115         }
116         handleEvent();
117         render();
118     }
119
120 void GUI::move() {
121
122     generation_count++;
123     setGenerationCountText();
124
125     std::set<Position2D>positions = space->move();
126     for (const Position2D &pos : positions)
127         grid->setGridColor(pos.x, pos.y, colors[space->getSpace
128             ()[pos.x][pos.y].state]);
129
130     graphic->updateFunction();
131 }
132
133 void GUI::clear() {
134     if (isRunning())
135         switchRunning();
136
137     btn_start->setText("Start");
138
139     space->clear();
140     move();
141 }
```

```

140     resetGenerationCount();
141
142     graphic->clear();
143 }
144
145 void GUI::initColors() {
146     colors.clear();
147
148     std::cout << "Size: " << rule_size << "\n";
149
150     for (int i = 0; i < rule_size; i++)
151         colors.push_back(generateRandomColor());
152
153     graphic->setColors(colors);
154 }
155
156 sf::Color GUI::generateRandomColor() {
157     // Inicializar la semilla aleatoria (solo una vez en el
158     // programa)
159     static bool initialized = false;
160     if (!initialized) {
161         srand(static_cast<unsigned>(time(nullptr)));
162         initialized = true;
163     }
164
165     // Generar valores RGB aleatorios
166     sf::Uint8 red = static_cast<sf::Uint8>(rand() % 256);
167     sf::Uint8 green = static_cast<sf::Uint8>(rand() % 256);
168     sf::Uint8 blue = static_cast<sf::Uint8>(rand() % 256);
169
170     return sf::Color(red, green, blue);
171 }

```

Listing 10: GUI.cpp

7.2.2. Grafica

```

1 void MainGraphic::updateFunction() {
2     for (int i = 0; i < functions.size(); i += 2) {
3         int count = 0;
4         std::vector<std::vector<Cell>> space = gui->space->
           getSpace();
5
6         for (const std::vector<Cell> &line : space)

```

```
7      for (const Cell &cell : line) {
8          if (cell.state == i / 2)
9              count++;
10     }
11
12     if (count > max) {
13         max = count;
14         gui->setMaxGenerationText(max);
15     }
16
17     int x = gui->getGenerationCount();
18     this->rescale(sf::Vector2f(0, 0), sf::Vector2f(x,
19         max));
20     functions[i]->addPoint(Vector2f(x, count), false);
21
22     functions[i + 1]->addPoint(Vector2f(x, log2(count)),
23         false);
24 }
25
26 void MainGraphic::clear() {
27     std::vector<sf::CircleShape> dots;
28     std::vector<sf::VertexArray> lines;
29
30     for (const std::shared_ptr<GraphicFunction> &function :
31         functions)
32         function->clear();
33
34     max = 3;
35 }
```

Listing 11: MainGraphic

7.2.3. Funcionamiento de botones

```
1 void ButtonStart::functionButton() {
2     gui->switchRunning();
3     if (gui->isRunning())
4         this->text.setString("Stop");
5     else
6         setText("Start");
7 }
8
9 void ButtonStep::functionButton() { gui->move(); }
```

```
10
11 void ButtonReset::functionButton() {
12     gui->clear();
13
14     gui->initColors();
15 }
16
17 void ButtonState::functionButton() {
18     std::string rule_string = gui->getRule();
19     std::vector<Rule> rules = stringToRules(rule_string);
20
21     gui->setRuleSize(rules.size());
22     gui->initColors();
23 }
24
25 void ButtonAnt::functionButton() {
26     gui->setDirection((gui->getDirection() + 1) % 5);
27     switch (gui->getDirection()) {
28         case 0:
29             this->setText("Add ant");
30             break;
31         case 1:
32             this->text.setString("UP");
33             break;
34         case 2:
35             this->text.setString("RIGHT");
36             break;
37         case 3:
38             this->text.setString("DOWN");
39             break;
40         case 4:
41             this->text.setString("LEFT");
42             break;
43     }
44 }
45
46 void ButtonColony::functionButton() {
47     gui->switchColony();
48     if (gui->isColony())
49         this->text.setString("Position...");
50     else
51         this->text.setString("Insert colony");
52 }
53
54 void TextBoxRule::functionButton(const std::string &input) {
```

```
55     int size = input.size();
56     gui->setRule(input);
57     gui->setRuleSize(size);
58     gui->space->setSpace(size);
59     gui->initColors();
60 }
61
62 void TextBoxRadio::functionButton(const std::string &input)
63 { gui->setRadio(std::stoi(input)); }
64
65 void TextBoxSize::functionButton(const std::string &input) {
66     gui->setSize(std::stoi(input)); }
67
68 void MainGrid::functionGrid(int x, int y) {
69     // insert colony
70     if(gui->isColony()) {
71         std::string rule_string = gui->getRule();
72         std::vector<Rule> rules = stringToRules(rule_string);
73         ;
74
75         gui->setCurrentColonies(gui->getCurrentColonies() +
76             1);
77         gui->space->insertColony(Position2D(x, y), gui->
78             getRadio(), gui->getSize(), rules, gui->
79             getCurrentColonies());
80         gui->move();
81
82         return;
83     }
84
85     // Insert one ant
86     int direction_int = gui->getDirection();
87     if (direction_int) {
88         Direction direction;
89         switch (direction_int) {
90             case 1:
91                 direction = Direction::UP;
92                 break;
93             case 2:
94                 direction = Direction::LEFT;
95                 break;
96             case 3:
97                 direction = Direction::DOWN;
98                 break;
```

```

94         case 4:
95             direction = Direction::RIGHT;
96             break;
97     }
98
99     std::string rule_string = gui->getRule();
100     std::vector<Rule> rules = stringToRules(rule_string)
101         ;
102
103     std::shared_ptr<Ant> ant = std::make_shared<Ant>(
104         Position2D(x, y), Position2D(this->width, this->
105             height),
106         direction, rules);
107
108     ant->display();
109
110     gui->space->insertAnt(ant);
111
112     this->rectangles[x][y]->setFillColor(Color::Red);
113 } else if (gui->space->getSpace()[x][y].ant.has_value())
114 {
115     gui->space->removeAnt(Position2D(x, y));
116     this->rectangles[x][y]->setFillColor(Color::Black);
117 }
118 }

```

Listing 12: ButtonFunctions

Los elementos gráficos como los botones, frames, campos de texto, cuadrículas, etc. que se extienden del proyecto *Graphic Mode* se omiten, únicamente se añaden aquellos elementos principales que interactúan directamente con la lógica.